bitfence: a pre-transaction risk layer for autonomous on-chain agents

Dennis Wecker, bitfence

Abstract

Autonomous AI agents now hold wallets, pay for services over x402, and execute on-chain transactions without a human in the loop. The risk tooling available to them was built for human readers: dashboards, explorers, and wallet warnings that assume a person will look at a screen and decide. bitfence is a risk oracle designed for the agent itself. It runs one check before every transaction that touches a token, returns a machine-routable verdict — a score, a recommendation, and named safety flags — and is itself paid for per call over x402, so an agent can discover, pay for, and consume the check without an account.

This paper explains the thinking behind the system. Risk is decomposed into six categories, where each category is a distinct mechanism by which a holder's position is destroyed. Safety-critical facts are treated with an explicit evidence discipline: every fact is known-true, known-false, or unknown, and an unknown is never quietly converted into a known. Hard safety checks are circuit breakers that override the composite score, split into measured threats and heuristic inferences with different authority. When the system cannot settle a safety-critical check, it says so, names the gap, and refuses to recommend proceeding.

---

1. The problem

An autonomous agent that holds a wallet will, sooner or later, transact against a token it has never seen. It may be swapping, paying, staking, or rebalancing a treasury. In every case the same question comes first: is this asset safe to handle at all?

Humans answer this question with tools built for humans. A block explorer renders history into tables a person can scan. A token-security dashboard shows labels — "honeypot", "high tax", "owner can mint" — in panels meant for a trader studying an unfamiliar token. A browser wallet interrupts with a warning when a person is about to click confirm. All of these assume a reader who scans, judges, and hesitates.

An agent does none of that. It calls a tool, receives structured fields, and routes on them. Wrapping a human dashboard in a REST endpoint does not close this gap, because the mismatch runs deeper than transport. The response shape is prose plus labels, which a model must interpret rather than route on. The billing model is accounts and API keys, which an autonomous wallet cannot open. And the failure behaviour is designed for a human who notices a blank panel — an agent consuming a score does not notice what is missing from it.

The timing constraint is absolute. A settled transaction cannot be unwound, so the check must complete before the transaction broadcasts, inside the latency budget of a single tool call. Monitoring after the fact serves reconciliation and telemetry; it does not protect the next transaction.

From this, four requirements:

1. **Interface.** The check must be callable inline as a tool. The response must have the same shape on every chain and every integration path, so the agent routes on fields instead of

interpreting prose.

2. **Economics.** The check must be payable per call, by the agent itself, without account creation — which is exactly the transaction shape x402 standardises.
3. **Interpretability.** The response must carry a score, an actionable recommendation (`PROCEED`, `REQUIRE_HUMAN_APPROVAL`, `BLOCK`), and hard-safety flags as explicit fields. A safety override folded invisibly into a composite number is an override the agent cannot act on.
4. **Fail-safe degradation.** When an upstream source is down, the system must say so, and the outage must not silently disable the one check that would have caught a scam. This requirement shapes more of the design than any other, and most of Section 2 exists to satisfy it.

---

2. Scoring methodology

Risk scores are integers from 0 to 100. Zero is minimal risk; one hundred is maximum risk. The score maps to three bands, and each band carries a recommendation: proceed, require human approval, or block.

2.1 Six categories, defined by loss mechanism

The first design decision is taxonomic: what is a risk category? The answer bitfence commits to is that a category is a *mechanism of loss* — a distinct way the holder's position gets destroyed. A category is never a data source, and never an analysis technique. Machine learning is a technique, not a category. A vendor is a supplier, not a mechanism. This rule keeps the taxonomy stable while sources and techniques change underneath it.

The six mechanisms:

1. **Exit integrity.** Can a holder sell — now, and later? This covers honeypot behaviour, sell and transfer taxes, whether taxes can be changed after launch, and authority configurations that can freeze or block transfers. Blocked exit means total loss, and it is the most common trap.
2. **Supply integrity.** Can the contract itself manipulate supply or balances — mint new tokens, confiscate holdings, freeze accounts, or self-destruct? Total loss through dilution or confiscation.
3. **Liquidity integrity.** Can the market be withdrawn out from under the holder? This covers whether pooled liquidity is locked or burned, when locks expire, how deep the pool is, and who controls it. Total loss through a rug pull.
4. **Holder and insider risk.** Who holds the supply, and will they dump on you? Concentration, insider cohorts, and the age profile of holder wallets. Typically partial loss.
5. **Provenance.** Who deployed this token, and what is their record? Provenance is a prior rather than a direct mechanism, but deployer history is among the most predictive rug signals in the published literature.
6. **Market integrity.** Is the observed activity real? Wash-traded volume and manufactured momentum are a deception aimed precisely at agents that act on market signals.

The composite weights these mechanisms by expected severity of loss: the total-loss mechanisms — blocked exit, withdrawn liquidity, contract-level supply manipulation — carry the majority of the weight. The exact weights are tuning parameters and are not published.

2.2 The composite, and what happens when data is missing

Let C be the set of six categories with nominal weights $w_c > 0$, $\sum_{c \in C} w_c = 1$. An assessment produces, for each category, a possibly empty set of signal scores $s_{c,1}, \dots, s_{c,n_c} \in [0, 100]$. A category is *live* when it produced at least one signal; write $L \subseteq C$ for the live set.

Within a category, the worst signal wins:

$$s_c = \max_i \; s_{c,i}$$

because one confirmed trap is not diluted by five clean readings next to it. The composite is then the weighted average over live categories only, with the weight vector renormalized to L:

$$S \;=\; \frac{\sum_{c \in L} w_c \, s_c}{\sum_{c \in L} w_c}$$

A category with no signals — because a source was unavailable, a call failed, or the fact class does not exist on that chain — appears in neither numerator nor denominator. This renormalization has one purpose: an absent category must neither drag the score toward "safe" nor pin it to an arbitrary midpoint. S reflects only what was actually observed.

That deliberately leaves a second question open: is an assessment with missing data safe to *act* on? The score does not answer that. The evidence discipline does.

2.3 Evidence discipline

Every fact that feeds a hard safety check is three-valued, taking a value in $\{\top, \bot, ?\}$ — known-true, known-false, unknown. Breaker predicates over these facts are evaluated under Kleene's strong three-valued logic: a predicate that touches an unknown does not evaluate to false, it evaluates to unknown.

The distinction that matters is between a negative answer and no answer. A source that responds "this token has no liquidity pool" has produced knowledge — a known $\bot$. A source that times out has produced nothing — the fact is ?. The discipline is that ? is never defaulted to $\top$ or $\bot$: missing data cannot fire a safety check, and missing data cannot suppress one.

A safety check fires only when its predicate evaluates to $\top$. When it evaluates to ?, the check is *indeterminate*, and the whole assessment is delivered as **degraded**: the response names the sources that failed, and the recommendation is capped. Order the recommendations `PROCEED` $<$ `REQUIRE_HUMAN_APPROVAL` $<$ `BLOCK`; then a degraded assessment's recommendation is

$$r' = \max(r, \texttt{REQUIRE\_HUMAN\_APPROVAL})$$

— it can never be `PROCEED`. Note the asymmetry against Section 2.2: the *score* stays fail-open (dark categories drop out and weights renormalize), while the *recommendation* fails safe. An upstream outage may narrow what the score can see, but it must never wave a token through on the strength of the checks it skipped.

The cap follows indeterminate checks, not dark sources. If a source is down but every safety check still evaluates to a known value from the remaining evidence, then nothing was disabled, and no cap applies. Degradation is reported whenever it exists and enforced only when it matters.

Most facts have more than one source, so the discipline extends across sources. When two sources report known values x_a, x_b for the same fact and $d(x_a, x_b) > \tau$ for that fact's tolerance τ, the fact is marked **contested**: it stops counting toward confidence and is named in the response. Disagreement is surfaced, never averaged away. And a safety check may fire on any source's known value, not only the primary's — a threat measured by a cross-validator is still a measured threat.

2.4 Circuit breakers

Circuit breakers are safety overrides evaluated after the composite. Each breaker b has a score floor ϕ_b; with F the set of fired breakers, the final score is

$$S_{\text{final}} = \max(S,\ \max_{b \in F} \phi_b)$$

A fired breaker raises the score to its floor; it never lowers a score already above it. Breakers are evaluated independently, and every fired breaker is named in the response as a structured flag.

Every breaker belongs to one of two classes, and the class is part of the breaker's definition, not a judgment made at the call site.

Measured-threat breakers fire on direct measurement of an active threat. A sell path that demonstrably reverts. A self-destruct path found in the bytecode. A measured, confiscatory sell tax. These override everything, including trusted-token status, because "a major stablecoin that measurably cannot be sold" means a wrong address or a live compromise — either way, block.

Heuristic breakers fire on inference. A dangerous authority configuration, extreme holder concentration, unlocked liquidity in the deployer's wallet, an external rug verdict. For an unknown token these are strong evidence of danger and they floor the score. But they never override trusted-token status, because the same configuration can be legitimate by design: the issuer of a major stablecoin holds mint and freeze authority deliberately.

The final verdict resolves in a fixed precedence order, highest authority first: measured-threat breakers, then the trusted-token cap, then heuristic breakers, then the degraded-assessment cap, then the composite.

The full set of breaker flags is part of the API contract and evolves with the policy engine, so it is not enumerated here. The trigger thresholds and the floors ϕ_b are not published anywhere. Publishing them would hand adversarial deployers a target: engineer the token to sit just below the trigger. The response tells the calling agent which checks fired; it does not tell a deployer where the trip-wires sit.

2.5 Maturity

A naive scorer punishes a ten-year-old token for the same traits a two-hour-old scam shows: retained mint authority, unlocked liquidity, a tax switch that was never used. Three mechanisms handle this asymmetry.

First, a curated **trusted-token registry** of established assets — major stablecoins, wrapped native assets, well-known governance tokens. A registry token's score is capped in the low band and it is exempt from heuristic breakers, because its "dangerous" configuration is documented design. It is never exempt from measured-threat breakers: the registry vouches for a design, not for whatever is live at an address today. The registry is versioned with the policy itself — its contents are hash-pinned, and any edit is a verdict-semantics change that cannot ship without an explicit policy-version bump.

Second, a softer computed test: when a token's primary pool is known to have existed beyond a maturity threshold, the authority-related signals are downgraded and the maturity-conditioned heuristic breakers are disabled.

Third, a stricter computed test: known evidence of long survival — a substantially older pool, or a very broad holder base — additionally exempts a token from the liquidity-related heuristic breakers.

Computed relief follows the same evidence discipline as everything else: it is granted on known facts only. A token whose pool age cannot be determined is neither mature nor immature — the fact is ?, the dependent checks go indeterminate, and the assessment degrades. Missing data never buys an exemption, and never manufactures a threat. The thresholds are unpublished, for the same reason breaker triggers are.

2.6 Confidence

Every assessment carries a confidence value $\kappa \in [0, 1]$. It measures coverage, not correctness. For a given chain, let A be the set of fact-bearing units that are *applicable* there — fact classes that can exist on that chain at all — and let $G \subseteq A$ be the units that answered fully and without contest during this assessment. Then

$$\kappa = \delta \cdot \frac{|G|}{|A|}$$

where $\delta \in (0, 1]$ is a per-chain structural discount, equal to 1 for most chains.

Two boundary rules keep the number honest. A known negative is an answer — "no pool exists" grounds the liquidity unit as completely as pool data would, so it counts toward G. And inapplicable fact classes are outside A entirely, so a chain is not permanently penalised for facts that cannot be measured there. The discount δ exists for chains whose market structure systematically limits what on-chain reads can observe: it preserves the ordering between that chain's assessments while marking all of them as partial views.

Confidence is reporting, not enforcement. The safety consequence of missing data is enforced server-side by the degradation cap of Section 2.3, so a caller does not need to interpret a float correctly to stay safe. The float remains useful as a graded signal: an operator can log it, alert on it, or hold a stricter line than the server enforces.

---

3. Data architecture

The scoring engine is chain-agnostic. Per-chain adapters feed it typed facts, and adding a chain is adapter work, not scoring work. Which chains are live at any moment is answered by the service itself, not by this document.

Facts reach the engine from three kinds of source, distinguished by epistemic role rather than by supplier:

1. **Direct measurement.** bitfence's own reads against the chain. On Solana-family chains this means parsing token accounts directly: mint and freeze authority, token-extension configuration, metadata mutability. On EVM-family chains it means bytecode and storage probes — self-destruct scanning, proxy detection, liquidity-burn checks — and the decisive exit measurement: a transaction simulation that routes a buy and a sell through the chain's own DEX router and observes whether, and at what cost, the sell completes. The safety-critical exit facts are measured first-hand, not purchased as verdicts.
2. **Indexed observation.** Aggregated views of market structure that no single chain read can produce: liquidity depth and lock schedules, holder distribution and cohorts, deployer history, and the market-activity series behind the market-integrity signals.
3. **Independent corroboration.** Additional independent services whose readings corroborate or contest facts from the first two roles. A corroborating source can ground a fact that observation missed, contest one it got wrong, and even fire a breaker on a known threat. It is advisory in the operational sense: no single corroborating source can veto an assessment on its own.

The suppliers behind these roles are deliberately not named, and the mapping from role to supplier is not one-to-one or fixed. Facts carry provenance internally, and any supplier can be replaced without changing what a fact means or how a signal scores. A supplier list in a public document would be stale within a quarter and useful mainly to competitors.

Structural gaps are named rather than papered over. Some fact classes cannot be measured on some chains — no sell-path simulation exists for Solana's execution model anywhere in the surveyed field, so the honeypot fact stays ? there, and the degradation reporting of Section 2.3 carries that gap to the caller. The discipline is uniform: where no source exists, the fact is unknown. It is never defaulted.

Operationally, adapters fan out concurrently with a hard deadline on every source call, so one stalled source degrades one fact rather than the whole request. Every fact records which sources observed it. By construction, every fact in an assessment is either grounded in a source that answered or explicitly ? because one did not.

---

4. System architecture

bitfence is a single service exposing one scoring engine through multiple interfaces: an HTTP/JSON API, a hosted MCP (Model Context Protocol) server for tool-calling agents, and in-process bindings for agent frameworks. Every interface returns the same assessment structure. The invariant worth stating is that the *contract* is standardised, not the transport: adding an interface means one more adapter at the edge delegating to the same analyzer, never a rewrite.

All scoring decisions — composite assembly, evidence grading, breaker evaluation, precedence — live in a pure policy engine with no I/O of its own. Purity is not an aesthetic choice; it is what makes the safety behaviour testable. Outage scenarios that are impractical to reproduce against live chains — this source dark, that fact contested, a breaker indeterminate — are exercised as deterministic tests over typed inputs. The policy carries an explicit version, and every assessment is stamped with the version that produced it.

A two-tier cache sits in front of the analyzer: in-process first, shared second. Its TTLs are keyed to the risk score itself, monotonically:

$$S_1 \leq S_2 \;\Rightarrow\; \mathrm{TTL}(S_1) \geq \mathrm{TTL}(S_2)$$

— the higher the risk, the shorter the cached life. The rationale is adversarial: if an upstream source is briefly manipulated into producing a distorted score, the window in which that score can persist is bounded, and bounded most tightly exactly where being wrong is most expensive. Three further rules keep the cache honest. A warm hit re-derives its verdict from the stored facts under the currently running policy, so a policy change takes effect without waiting for expiry. An entry stamped with an older policy version is treated as a miss, so no verdict outlives the policy that produced it. And degraded assessments are never written to the shared cache, so an outage is re-checked on every request instead of being served as if it were a settled answer.

Payment follows the x402 pattern: HTTP's 402 status carried to completion. A request without payment receives a `402 Payment Required` response whose body contains the payment challenge and machine-readable discovery metadata — name, description, schema, example input and output — so an agent can discover, understand, pay for, and call the service without a human registering anywhere. The caller retries with a signed payment header; the server verifies it, runs the assessment, and settles on-chain only if the assessment succeeded. A failed assessment is never charged, and a response is never delivered against a failed settlement.

Assessments can also be position-aware. Given a position size and portfolio context, the same engine layers execution risk — pool depth, expected slippage, concentration — on top of the base verdict. The rule that matters is directional, and it is the same escalation-only rule as the degradation cap: context can raise a recommendation in the `PROCEED` < `REQUIRE_HUMAN_APPROVAL` < `BLOCK` ordering, never lower it. A blocked token stays blocked at any position size.

References

1. **x402 protocol specification.** Coinbase, `coinbase/x402` repository. https://github.com/coinbase/x402
2. **Model Context Protocol specification.** Anthropic. https://spec.modelcontextprotocol.io/
3. **Xia et al.** "Trade or Trick? Detecting and Characterizing Scam Tokens on Uniswap Decentralized Exchange." *ACM Internet Measurement Conference (IMC)*, 2021.
4. **Mazorra, Adan, Daza.** "Do Not Rug on Me: Leveraging Machine Learning Techniques for Automated Scam Detection." *Mathematics*, vol. 10, no. 6, 2022.
5. **Cernera et al.** "Token Spammers, Rug Pulls, and Sniper Bots: An Analysis of the Ecosystem of Tokens in Ethereum and in the Binance Smart Chain (BNB)." *USENIX Security Symposium*, 2023.
6. **Daian et al.** "Flash Boys 2.0: Frontrunning, Transaction Reordering, and Consensus Instability in Decentralized Exchanges." *IEEE Symposium on Security and Privacy*, 2020.